

PDFBooker

Transforms websites into beautifully rendered and readable PDF Books

PDFBooker (pdfbooker.xyz) transforms web content into beautifully formatted PDF books with just a few clicks. Users can enter any URL and specify how deep they want the crawler to go, and the system automatically scrapes all connected pages, cleans up the content to remove ads and unnecessary elements, and generates a professional-looking PDF "book" with proper formatting, table of contents, and navigation. Built with a sleek Shadcn UI interface, the tool makes it easy to customize the final output with options for title pages, fonts, and layout styles. Perfect for creating offline reading collections, archiving research materials, or generating documentation from web resources.

Phase 1: Project Setup & Architecture

Action Item 1: Set up a web application with Shadcn UI

Create a new web application project that uses Shadcn UI components:

1. Initialize a Next.js or React project
2. Install and configure Tailwind CSS
3. Set up Shadcn UI (<https://ui.shadcn.com/docs/installation>)
4. Create the basic file structure:
 - pages/ or app/ directory for routes
 - components/ directory for UI components
 - lib/ directory for utility functions
 - server/ directory for backend code
5. Set up a basic layout with Shadcn theming

Action Item 2: Design the application architecture

Design the overall architecture for the web scraper to PDF application:

1. Frontend components using Shadcn UI for user input and progress display
2. Backend server for web scraping (to avoid CORS issues)
3. Content processing pipeline (scraping, cleaning, formatting)
4. PDF generation module
5. Data flow between components

Provide a diagram or detailed explanation of how these components will interact.

Phase 2: Frontend Development with Shadcn UI

Action Item 3: Create the user interface with Shadcn components

Develop a clean, responsive UI using Shadcn UI components:

1. Use Card component for the main input form
2. Implement Input component for the URL entry
3. Use Slider or Select component for depth selection
4. Add Button component for the start and download actions
5. Implement Progress component for tracking operations
6. Use Tabs for organizing different sections (URL input, PDF customization)
7. Add Form components with validation
8. Implement Accordion for advanced options

Make sure the UI is responsive and follows Shadcn's design system.

Action Item 4: Implement form validation with Shadcn UI form components

Set up form validation using Shadcn UI's form components:

1. Implement React Hook Form integration as recommended by Shadcn UI
2. Set up zod schema for validation of:
 - URL format (valid URL structure)
 - Depth parameter (numeric, within reasonable range)
 - PDF customization options
3. Add appropriate error messages using Shadcn's form error states
4. Implement Toast notifications for submission feedback
5. Add loading states to buttons during form submission

Phase 3: Backend Development

Action Item 5: Set up a Node.js server

Create a Node.js server with Express that will:

- Handle API requests from the Shadcn UI frontend
- Manage the web scraping process
- Process and store scraped content
- Return the final PDF to the client

Include appropriate error handling, CORS configuration, and logging.

Action Item 6: Implement web scraping functionality

Develop the web crawler module that:

- Takes a URL and depth as input
- Recursively scrapes linked pages up to the specified depth
- Handles different content types and site structures
- Respects robots.txt and implements rate limiting
- Stores the scraped content in an organized way
- Provides status updates that can be sent to the frontend

Use libraries like Puppeteer or Cheerio for scraping.

Phase 4: Content Processing

Action Item 7: Create a content cleaner/parser

Build a module that processes the scraped HTML content:

- Extracts the main content using readability algorithms
- Removes ads, navigation, footers, and other non-content elements
- Preserves important images and formatting
- Normalizes and structures the content for PDF generation
- Maintains proper heading hierarchy and document structure

Use libraries like Mozilla's Readability or similar tools.

Action Item 8: Implement link resolution and content organization

Create functionality to:

- Resolve relative links within scraped content
- Download and include necessary images
- Organize content in a logical reading order

- Generate a table of contents based on the crawled pages
- Create proper transitions between different pages in the final book
- Design a structure that will work well with PDF generation

Phase 5: PDF Generation

Action Item 9: Develop PDF generation capability

Implement a module that converts the processed content into a well-formatted PDF:

- Create a clean, professional book layout
- Include cover page, table of contents, and page numbers
- Support custom styling options (fonts, colors, margins)
- Handle images properly with appropriate sizing and resolution
- Generate bookmarks for different sections

Use libraries like PDF.js, jsPDF, or html-pdf for PDF generation.

Action Item 10: Add PDF customization options with Shadcn UI components

Create features that allow users to customize their PDF output using Shadcn UI:

1. Use Input components for book title and author information
 2. Implement ColorPicker for theme customization
 3. Add Select dropdowns for font family and size options
 4. Use Slider for margin and spacing adjustments
 5. Implement Radio or Checkbox groups for layout options
 6. Add FileInput for custom cover image upload
 7. Use Shadcn's Switch component for toggling features
- Ensure all these settings are properly applied to the PDF generation process.

Phase 6: Integration & User Experience

Action Item 11: Connect Shadcn UI frontend with backend

Integrate the Shadcn UI frontend with the backend API:

1. Create API client functions to communicate with the server
2. Use React Query or SWR for data fetching and state management
3. Implement proper error handling with Shadcn's Alert and Toast components
4. Show real-time progress using Shadcn's Progress component
5. Use Skeleton components for loading states
6. Add Dialog components for confirmation actions
7. Implement error boundaries with fallback UI

Use Axios or fetch API for API communication.

Action Item 12: Implement progress tracking with Shadcn UI

Build a system to track and display the progress using Shadcn UI components:

1. Use Progress component to show overall completion percentage
2. Implement Step or custom Stepper component to show current stage
3. Add Badge components to show status of different processes
4. Use Accordion to display detailed progress information
5. Implement Toast notifications for important milestones
6. Add hover tooltips for additional information

Ensure all progress updates are visually consistent with Shadcn's design system.

Phase 7: Testing & Refinement

Action Item 13: Test with various websites

Test the application with different types of websites:

- News sites with many articles
- Blogs with various content structures
- Documentation sites with complex navigation
- Sites with different image handling
- Sites with pagination and dynamic content

Document any issues found and prioritize fixes, especially any rendering issues with Shadcn UI components.

Action Item 14: Optimize performance and reliability

Improve the application's performance and reliability:

- Implement caching for scraped content
- Add retry mechanisms for failed requests
- Optimize image processing to reduce PDF size
- Add memory management for large crawls
- Implement timeout handling for slow sites
- Ensure Shadcn UI components render efficiently

Phase 8: Deployment & Final Touches

Action Item 15: Prepare for deployment

Get the application ready for deployment:

1. Configure build process for the Shadcn UI frontend
2. Set up proper environment configuration
3. Implement security best practices
4. Create documentation for users with Shadcn UI components
5. Add dark mode support using Shadcn's theming
6. Ensure accessibility compliance
7. Test cross-browser compatibility
8. Add final visual polish to the UI

Written with [StackEdit](#).